

DFI PHY Input Interface

Complete Signal Reference — All Groups

Control · Write Data · Read Data · Update · Status · Training · Low Power

Date: March 7, 2026

About This Document

This document provides a complete, detailed analysis of every signal in the DFI (DDR PHY Interface) PHY Input Interface. The DFI standard defines a digital bus between the Memory Controller (MC) and the PHY. Signals are grouped into seven functional interfaces: Control, Write Data, Read Data, Update, Status, Training, and Low Power. The suffix `_pN` indicates per-slice (per-byte-lane) signals, where N is the slice index. All signal directions are MC $\rightarrow$ PHY unless noted otherwise.

Contents

1	DFI Architecture Overview	2
1.1	What is DFI?	2
1.2	Signal Direction Convention	2
1.3	The _pN Per-Slice Suffix	2
1.4	Full Interface Block Diagram	3
2	Control Interface	3
2.1	d_address_pN	3
2.2	d_bank_pN	4
2.3	d_cas_n_pN	4
2.4	d_cke_pN	4
2.5	d_cs_n_pN	5
2.6	d_odt_pN	5
2.7	d_ras_n_pN	6
2.8	d_reset_n_pN	6
2.9	d_we_n_pN	6
3	Write Data Interface	7
3.1	d_wrdata_pN	7
3.2	d_wrdata_en_pN	7
3.3	d_wrdata_mask_pN	8
4	Read Data Interface	8
4.1	d_rddata_en_pN	9
5	Update Interface	9
5.1	d_ctrlupd_req	9
5.2	d_phyupd_ack	10
6	Status Interface	10
6.1	d_data_byte_disable	11
6.2	d_dram_clk_disable	11
6.3	d_freq_ratio	11
6.4	d_init_start	12
6.5	d_parity_in_pN	12
7	Training Interface	13
7.1	Read Levelling Signals	13
7.1.1	d_rdlvl_load	13
7.1.2	d_rdlvl_cs_n	13
7.1.3	d_rdlvl_en	13
7.1.4	d_rdlvl_edge	14
7.1.5	d_rdlvl_delay_X	14
7.1.6	d_rdlvl_gate_en	15
7.1.7	d_rdlvl_gate_delay_X	15
7.2	Write Levelling Signals	15
7.2.1	d_wrlvl_load	15
7.2.2	d_wrlvl_cs_n	16
7.2.3	d_wrlvl_strobe	16
7.2.4	d_wrlvl_en	16
7.2.5	d_wrlvl_delay_X	17

8	Low Power Control Interface	17
8.1	d_lp_req	17
8.2	d_lp_wakeup	17
9	Complete Signal Quick-Reference Table	18
10	Signal Interaction: Key Timing Diagrams	20
10.1	Write Transaction Timing	20
10.2	Init + Training Handshake	21
11	Summary and Key Takeaways	21

1 DFI Architecture Overview

1.1 What is DFI?

Key Concept 1.1: DDR PHY Interface (DFI)

DFI is an industry-standard specification (maintained by the DFI Workgroup) that defines the digital interface between a Memory Controller (MC) and a DDR PHY. Its primary goal: make the MC PHY-agnostic and the PHY controller-agnostic. Any DFI-compliant MC can connect to any DFI-compliant PHY without modification.

Current versions: DFI 2.1 (DDR3), DFI 3.1 (DDR4), DFI 5.0 (DDR5).

1.2 Signal Direction Convention

Table 1: DFI Signal Direction Summary

Interface	Primary Direction	Driver	Receiver
Control Interface	MC → PHY	Memory Controller	PHY
Write Data Interface	MC → PHY	Memory Controller	PHY
Read Data Interface	PHY → MC	PHY	Memory Controller
Update Interface	Bidirectional	Both	Both
Status Interface	Both directions	Both	Both
Training Interface	Bidirectional	Both	Both
Low Power Interface	Bidirectional	Both	Both

1.3 The _pN Per-Slice Suffix

Key Concept 1.2: Per-Slice Signals

Signals with the suffix _pN exist as one copy per byte-lane (slice). For a 72-bit DDR4 channel (9 byte lanes = 8 data + 1 ECC):

$$N \in \{0, 1, 2, \dots, 8\}$$

This means `dfi_wrdata_pN` is actually 9 independent buses: `dfi_wrdata_p0`, `dfi_wrdata_p1`, ..., `dfi_wrdata_p8`. Each carries 8 bits (one per DQ pin in the lane) for 2 DFI cycles ($= 1 t_{CK}$), making the total data bus width $9 \times 8 \times 2 = 144$ bits per DFI cycle pair.

1.4 Full Interface Block Diagram

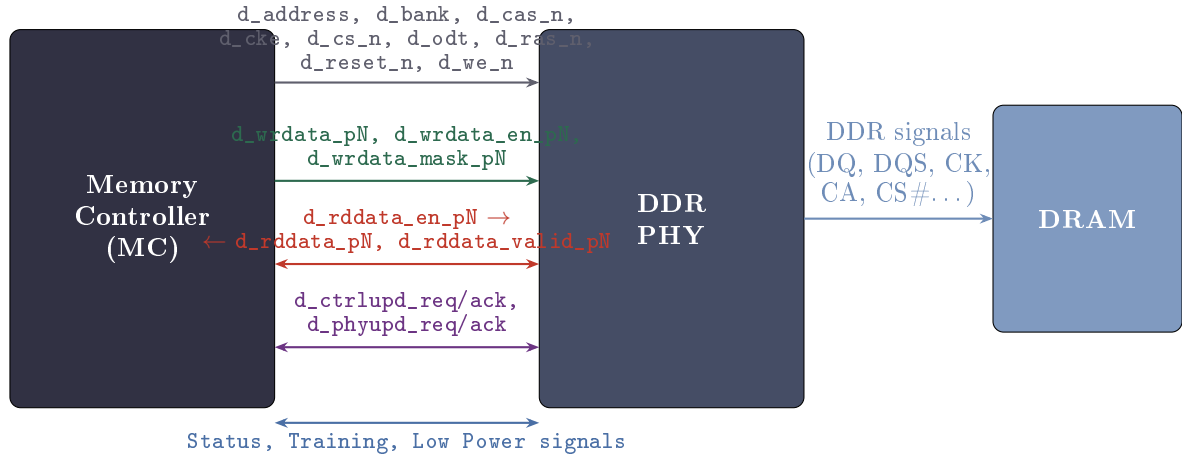


Figure 1: DFI Interface: all seven groups connecting Memory Controller to PHY, and PHY to DRAM.

2 Control Interface

Interface Group: Control Interface

The Control Interface carries address, bank, and command signals from the MC to the PHY, which then drives the corresponding DRAM pins. These signals are registered against the DFI clock and forwarded to the DRAM on the appropriate CK edge after a fixed pipeline latency.

Timing: All control signals are sampled on the DFI clock rising edge. The PHY applies an internal command latency (t_{cmd_lat}) before presenting them on the DRAM pins.

Direction: MC → PHY

2.1 d_address_pN

Signal: dfi_address_pN

Full Name: DFI Address

Direction: MC → PHY

Width: Per-slice; typically 14–18 bits per rank (DDR4: A[13:0] + A14/WE#/A15/CAS#/A16/RAS#)

Function: Carries the row/column address and mode register address to the PHY. The PHY serialises this onto the A[13:0] DRAM pins synchronised to CK. In DDR4, the same pins carry row address (ACT command) and column address (READ/WRITE command) at different times.

Key Behaviour:

- During **ACTIVATE**: carries row address RA[15:0] (spread across A, RAS#/A16, etc.)
- During **READ/WRITE**: carries column address CA[9:0] via A[9:0] and A[11]
- During **MRS**: carries mode register data across A[13:0]
- During **NOP/DES**: don't-care, but PHY still drives the pins to a defined level

Timing: Must be stable t_{IS} before and t_{IH} after CK at the DRAM. The PHY's CA TX delay line compensates for PCB skew via CA training.

2.2 d_bank_pN

Signal: dfi_bank_pN
Full Name: DFI Bank Address
Direction: MC → PHY
Width: 2 bits (DDR3/4: BA[1:0]), extended in DDR5
Function: Selects the target bank within the DRAM. Together with dfi_address, it fully specifies the memory row or column target.

DDR4 Bank Decode:

- BA[1:0] selects one of 4 banks within a bank group
- Bank Group (BG[1:0]) selects the bank group (DDR4 adds 2-level bank hierarchy)
- Combined: $4 \times 4 = 16$ banks total (DDR4)

Note: In some DFI implementations, bank group bits BG[1:0] are carried within dfi_address at higher bit positions rather than as a separate signal. Check DFI version.

2.3 d_cas_n_pN

Signal: dfi_cas_n_pN
Full Name: DFI Column Address Strobe (active-low)
Direction: MC → PHY
Width: 1 bit (per rank group)
Function: In DDR3, this is the CAS# pin — one of the three command pins (RAS#, CAS#, WE#) that together with CS# form the command truth table. In DDR4, CAS# is repurposed as A15 but the DFI signal is maintained for backward compatibility.

Command encoding with RAS#/CAS#/WE#:

Command	RAS#	CAS#	WE#	ACT#
NOP	H	H	H	H
Activate	L	H	H	L
Read	H	L	H	H
Write	H	L	L	H
Precharge	L	H	L	H
Refresh	L	L	H	H
MRS	L	L	L	H

2.4 d_cke_pN

Signal: dfi_cke_pN
Full Name: DFI Clock Enable
Direction: MC → PHY
Width: 1 bit per rank (or per DRAM device in some configs)

Function: Controls the DRAM power state machine via the CKE DRAM pin. It determines whether the DRAM is in an active, power-down, or self-refresh state.

CKE State Transitions:

- CKE=1, bank idle: Active idle state
- CKE=0, banks precharged: Precharge Power Down (PPD)
- CKE=0, banks active: Active Power Down (APD)
- CKE=0 after self-refresh command: Self-Refresh (SR)
- CKE=1 from SR: Self-Refresh Exit, triggers t_{XS} timer

Critical: The PHY must ensure CKE transitions respect t_{CKE} (minimum CKE pulse width) and $t_{XS DLL}$ (DLL lock time after SR exit). The MC sends CKE via DFI sufficiently early to account for the PHY's pipeline latency.

2.5 d_cs_n_pN

Signal: dfi_cs_n_pN

Full Name: DFI Chip Select (active-low)

Direction: MC → PHY

Width: 1 bit per rank

Function: This is the most critical command signal. The DRAM only latches any command (ACT, RD, WR, MRS, etc.) when CS_n is asserted LOW at the CK rising edge. When CS_n is HIGH, the DRAM ignores all other command pins (NOP / Deselect).

Multi-Rank Use: In a 2-rank DIMM: dfi_cs_n_p0[1:0] where bit[0] selects Rank 0 and bit[1] selects Rank 1. Only one CS# should be asserted per CK cycle in normal operation.

DFI Timing:

$$t_{CS_n, DRAM} = t_{CS_n, DFI} + t_{phy_cmd_lat} \times t_{CK}$$

where $t_{phy_cmd_lat}$ is the PHY's command pipeline depth (typically 1–3 cycles).

2.6 d_odt_pN

Signal: dfi_odt_pN

Full Name: DFI On-Die Termination

Direction: MC → PHY

Width: 1 bit per rank

Function: Controls the ODT pin of each DRAM rank. When asserted, the DRAM switches on its internal parallel termination resistors (R_{TT}) on DQ/DQS pins.

ODT Scheduling Rules:

- READ from Rank A: Assert ODT on Rank B (non-target), deassert on Rank A
- WRITE to Rank A: Assert ODT on Rank A (R_{TT_WR}), assert R_{TT_PARK} on Rank B
- Idle: Both ranks may use R_{TT_PARK} (background termination)

Critical Timing: The ODT assertion must lead the data burst:

- ODT turn-on propagation: $t_{AONPD} = 2 \times t_{CK}$
- ODT must be fully on before data arrives at DRAM

The MC calculates the DFI cycle on which to assert `dfi_odt` by subtracting the PHY pipeline depth and the t_{AONPD} latency from the target data cycle.

2.7 d_ras_n_pN

Signal: `dfi_ras_n_pN`

Full Name: DFI Row Address Strobe (active-low)

Direction: MC → PHY

Width: 1 bit (per rank group)

Function: In DDR3, RAS# is one of the three primary command decode pins. In DDR4, it is repurposed as A16 (the 17th address bit), but the DFI signal is preserved for DDR3 compatibility. Together with `dfi_cas_n` and `dfi_we_n`, it forms the complete 3-bit command word.

Note: In DDR5, the entire command/address bus concept changes to a serial CA bus. RAS#, CAS#, WE# are replaced by encoded 14-bit CA values. However, DFI 5.0 still maintains backward-compatible signals.

2.8 d_reset_n_pN

Signal: `dfi_reset_n_pN`

Full Name: DFI DRAM Reset (active-low)

Direction: MC → PHY

Width: 1 bit

Function: Drives the RESET# pin of all connected DRAM devices. This is the only asynchronous signal in the DDR interface:

- When asserted LOW: All DRAM outputs immediately go to Hi-Z regardless of CK state
- All internal DRAM state machines reset asynchronously
- All mode register settings return to default

Power-Up Sequence:

$t_{RESET} \geq 200 \mu s$ (RESET# must be held LOW for $\geq 200 \mu s$ after VDD stable)

The MC asserts `dfi_reset_n=0` during the power-up sequence and holds it until the full initialisation timer expires. The PHY must pass this through immediately (no pipeline delay) due to its asynchronous nature.

2.9 d_we_n_pN

Signal: `dfi_we_n_pN`

Full Name: DFI Write Enable (active-low)

Direction: MC → PHY

Width: 1 bit (per rank group)

Function: The third command pin in the DDR3 command triplet. In DDR4, repurposed as A14. Controls the WRITE vs. READ distinction in column-access commands:

- $WE\#=0$ (with $CAS\#=0$): WRITE command
- $WE\#=1$ (with $CAS\#=0$): READ command

Also selects between bank precharge options and MRS variants.

3 Write Data Interface

Interface Group: Write Data Interface

The Write Data Interface carries the actual write payload — data, enable strobes, and byte masks. These signals are per-slice (**pN**) because each byte lane is an independent data path with its own timing requirements. The PHY serialises the parallel DFI data into the high-speed DQ stream.

DFI Data Rate: Each DFI clock cycle carries 2 bits per DQ pin (one rising + one falling edge). For BL=8, the MC presents 8 DFI words (4 DFI clock cycles) per write burst.

Direction: MC → PHY

3.1 d_wrdata_pN

Signal: dfi_wrdata_pN

Full Name: DFI Write Data

Direction: MC → PHY

Width: $2 \times 8 = 16$ bits per slice (8 rising-edge bits + 8 falling-edge bits), sometimes expressed as $2 \times DQ_width$

Function: Carries the actual write data payload for one byte lane. The factor-of-2 accounts for DDR (Double Data Rate): within a single DFI clock cycle, the PHY serialises two bits to the DRAM.

- Bits [7:0]: Data to be driven on DQ[7:0] on the **rising** DQS edge
- Bits [15:8]: Data to be driven on DQ[7:0] on the **falling** DQS edge

Timing Relationship: The MC must present dfi_wrdata exactly t_{phy_wrlat} DFI clocks after the WRITE command on dfi_cs_n/dfi_cas_n:

$$\text{Cycle of dfi_wrdata} = \text{Cycle of WRITE command} + WL - t_{phy_wrlat}$$

where $WL = AL + CWL$ (Write Latency = Additive Latency + CAS Write Latency).

3.2 d_wrdata_en_pN

Signal: dfi_wrdata_en_pN

Full Name: DFI Write Data Enable

Direction: MC → PHY

Width: 1 bit per slice (sometimes 2 bits for rising/falling separately)

Function: A qualifier signal that tells the PHY when `dfi_wrdata` is valid and should be forwarded to the DRAM. It acts as a burst enable window:

- **HIGH:** PHY drives DQ[7:0] with data from `dfi_wrdata`; PHY generates DQS output
- **LOW:** PHY tristates the DQ drivers; DQS is not generated (inter-burst silence)

Burst Duration: For BL=8, `dfi_wrdata_en` is asserted for 4 consecutive DFI clocks (= 8 DQS edges = 8 DQ bits per pin).

Timing 3.1: Write Data Enable Timing

Assert cycle = WRITE cmd cycle + $WL - t_{phy_wrlat}$

Deassert cycle = Assert cycle + $BL/2 - 1$ (BL/2 DFI clocks for BL=8: 4 clocks)

The PHY uses this enable to generate the DQS preamble (1 before first DQS edge) and postamble (0.5 after last DQS edge).

Critical: If `dfi_wrdata_en` is asserted without corresponding valid `dfi_wrdata`, the PHY will drive undefined data to the DRAM. The MC must always assert both together.

3.3 d_wrdata_mask_pN

Signal: `dfi_wrdata_mask_pN`

Full Name: DFI Write Data Mask

Direction: MC → PHY

Width: 2 bits per slice (1 for rising edge, 1 for falling edge, matching DM pin)

Function: Drives the DM (Data Mask) or DBI (Data Bus Inversion) DRAM pin. When a mask bit is asserted (=1), the corresponding byte is not written to the DRAM array (the write is suppressed for that byte on that clock edge).

Use Cases:

- **Byte-granular writes:** Write only specific bytes of a cache line without a read-modify-write cycle
- **ECC handling:** Mask certain bytes during ECC scrubbing
- **DBI mode (DDR4/5):** When DBI is enabled, this signal carries the inversion indicator instead of a mask

Note: In DDR5, `dfi_wrdata_mask` is sometimes called `DM_n` and operates per-nibble (4 bits per cycle per lane). The DBI logic may be computed inside the PHY when configured in DBI mode.

4 Read Data Interface

Interface Group: Read Data Interface — Bidirectional

The Read Data Interface is bidirectional. The MC sends a read enable control signal (input to PHY), and the PHY returns the captured data and a validity flag (output from PHY).

Direction: Bidirectional

4.1 d_rddata_en_pN

Signal: dfi_rddata_en_pN

Full Name: DFI Read Data Enable

Direction: MC → PHY

Width: 1 bit per slice

Function: Tells the PHY when to expect incoming read data from the DRAM so it can open the DQS gate, enable the deserialiser, and capture DQ bits. It is a *look-ahead enable*: the MC sends it t_{phy_rdlat} DFI cycles before the data actually arrives at the PHY's pins.

Assert cycle of `dfi_rddata_en` = READ cmd cycle + $RL - t_{phy_rdlat}$

What the PHY Does When It Receives This:

1. Opens the DQS gate (enables DQS-clocked capture flip-flops)
2. Starts the RX FIFO write pointer
3. After t_{phy_rdlat} cycles, the captured data is available on `dfi_rddata`

Protocol 4.1: Read Data Return Signals (PHY → MC)

These two signals are **outputs FROM the PHY** (not inputs from MC), but are part of the Read Data Interface and documented here for completeness:

d_rddata_pN (PHY → MC): The captured read data. Width = 2×8 bits per slice (rising + falling DQS edge data). The PHY presents this t_{phy_rdlat} DFI cycles after receiving `dfi_rddata_en`.

d_rddata_valid_pN (PHY → MC): Asserted HIGH for each DFI cycle in which `dfi_rddata` contains valid captured data. The MC must only consume read data when this qualifier is HIGH. This is crucial because the read latency can vary and the valid signal is the authoritative indicator.

5 Update Interface

Interface Group: Update Interface

The Update Interface provides a handshake mechanism for PHY and MC to safely apply internal configuration updates. Because the PHY has internal state (delay line codes, PLL settings, DLL lock state) that must be updated atomically with respect to ongoing transactions, a request/acknowledge protocol is needed.

Direction: Bidirectional handshake

5.1 d_ctrlupd_req

Signal: dfi_ctrlupd_req

Full Name: DFI Controller Update Request

Direction: MC → PHY

Width: 1 bit

Function: The MC asserts this signal to request that the PHY apply a pending configuration update. This could include new delay line codes (from re-training or periodic recalibration), new DLL settings after a frequency change, ZQ recalibration results, or mode register

write results that affect PHY timing.

Protocol:

1. MC asserts `dfi_ctrlupd_req` = 1
2. PHY waits for a safe window (no active burst), then applies the update atomically
3. PHY asserts `dfi_ctrlupd_ack` = 1 (PHY → MC) to signal completion
4. MC deasserts `dfi_ctrlupd_req` = 0
5. PHY deasserts `dfi_ctrlupd_ack` = 0

Maximum Hold Time: The PHY must complete the update and assert ACK within $t_{CTRLUPD_MAX}$. If it exceeds this, the MC can assume a PHY error.

5.2 d_phyupd_ack

Signal: `dfi_phyupd_ack`

Full Name: DFI PHY Update Acknowledge

Direction: MC → PHY (MC's response to a PHY-initiated update request)

Width: 1 bit

Function: This signal is the MC's acknowledgement to the PHY's own update request (`dfi_phyupd_req`, a PHY→MC signal). When the PHY needs to update its own internal state (e.g., re-lock a DLL, apply a ZQ recalibration, compensate for temperature drift), it raises `dfi_phyupd_req` and waits for the MC to assert `dfi_phyupd_ack` before proceeding.

Why This Is MC→PHY: The MC must ensure there are no pending transactions before granting the PHY permission to update. The ACK means “*I have paused all traffic, you may safely update.*”

Protocol:

1. PHY raises `dfi_phyupd_req` = 1 (PHY → MC)
2. MC completes any in-flight bursts, then asserts `dfi_phyupd_ack` = 1 (MC → PHY)
3. PHY performs its update (DLL re-lock, ZQ update, etc.)
4. PHY deasserts `dfi_phyupd_req` = 0
5. MC deasserts `dfi_phyupd_ack` = 0, resumes scheduling

6 Status Interface

Interface Group: Status Interface

The Status Interface carries configuration, mode, and initialisation signals. These are typically quasi-static: they change infrequently (at boot, on mode changes, or during power state transitions) rather than every clock cycle. Some signals flow from MC to PHY (configuration); others flow from PHY to MC (status reporting).

Direction: Mixed directions

6.1 d_data_byte_disable

Signal: dfi_data_byte_disable

Full Name: DFI Data Byte Disable

Direction: MC → PHY

Width: 1 bit per slice (byte lane)

Function: Permanently disables a specific byte lane (slice) in the PHY. When asserted:

- The PHY tristates the DQ[7:0] and DQS/DQS# drivers of that lane
- The lane is excluded from training
- Useful for ×16 configuration in a ×64 channel (disable lanes not connected to DRAM pins)
- Also used during diagnostic testing to isolate a faulty lane

This is a **static configuration signal** set once during initialisation and not changed during normal operation.

6.2 d_dram_clk_disable

Signal: dfi_dram_clk_disable

Full Name: DFI DRAM Clock Disable

Direction: MC → PHY

Width: 1 bit per rank (or 1 global bit)

Function: Instructs the PHY to stop driving the CK/CK# differential clock to the DRAM. Used in Self-Refresh, Power Gating, and Clock Frequency Change scenarios.

Critical Sequence:

1. MC issues SELF-REFRESH command via dfi_cs_n/dfi_ras_n/dfi_cas_n
2. MC deasserts CKE via dfi_cke = 0
3. MC waits $\geq t_{CKSRE}$ (DRAM clock stable hold time after CKE low)
4. MC asserts dfi_dram_clk_disable = 1
5. PHY stops CK toggling cleanly (avoids partial clock cycle)

6.3 d_freq_ratio

Signal: dfi_freq_ratio

Full Name: DFI Frequency Ratio

Direction: MC → PHY (configuration)

Width: 2 bits

Function: Specifies the ratio of the DRAM clock frequency to the DFI clock frequency. This is critical for the PHY to correctly map DFI cycles to DRAM CK cycles.

d_freq_ratio	Ratio	Meaning
2'b01	1:1	DFI clock = DRAM CK (full-rate mode, rare)
2'b10	1:2	DFI clock = DRAM CK / 2 (half-rate, common DDR4)
2'b11	1:4	DFI clock = DRAM CK / 4 (quarter-rate, DDR5 at high speeds)

Impact on Timing Parameters: All DFI timing parameters (e.g., t_{phy_wrlat} , t_{phy_rdlat}) are expressed in DFI clocks. Their values depend on **dfi_freq_ratio**:

$$t_{phy_wrlat,DFI} = \left\lfloor \frac{t_{phy_wrlat,CK}}{freq_ratio} \right\rfloor$$

6.4 d_init_start

Signal: dfi_init_start

Full Name: DFI Initialisation Start

Direction: MC → PHY

Width: 1 bit

Function: A level-sensitive trigger that instructs the PHY to begin its DRAM initialisation and training sequence. This is the MC's way of saying *"I am ready, please start the PHY/DRAM init procedure."*

Full Boot Sequence:

1. System power stabilises; voltage rails within spec
2. MC holds **dfi_init_start** = 0; PHY performs PLL lock, DLL lock
3. MC asserts **dfi_init_start** = 1
4. PHY drives RESET# LOW, applies initialisation delays, then brings DRAM out of reset
5. PHY performs ZQ calibration, write levelling, read training, etc.
6. PHY asserts **dfi_init_complete** = 1 (PHY → MC) when training is done
7. MC deasserts **dfi_init_start**; begins normal memory scheduling

*Note: The entire training sequence (100–500 μs) happens while **dfi_init_start** is held HIGH.*

6.5 d_parity_in_pN

Signal: dfi_parity_in_pN

Full Name: DFI Command/Address Parity In

Direction: MC → PHY

Width: 1 bit per slice / rank

Function: Carries the computed odd parity bit over the CA bus for DDR4/5 DRAM devices that support command address parity (CAP). The PHY forwards this to the DRAM's PAR pin.

$$PAR = \bigoplus (A[13:0] \oplus BA[1:0] \oplus BG[1:0] \oplus ACT\# \oplus RAS\# \oplus CAS\# \oplus WE\#) \oplus 1$$

The XOR of all CA bits plus a constant 1 ensures odd parity (always an odd number of 1s including PAR itself).

Error Response: If the DRAM detects a parity error, it asserts ALERT# (active low). The PHY feeds this back to the MC via a status signal. The MC must then retry the errored command.

7 Training Interface

Interface Group: Training Interface

The Training Interface is the most complex DFI interface group. It provides fine-grained, per-byte-lane control for the PHY's training state machines: read levelling (read gate training + centering) and write levelling. All training signals are bidirectional and per-slice (pN). The MC drives the training control signals; the PHY drives training status/result signals back.

Direction: Bidirectional, per-slice

7.1 Read Levelling Signals

7.1.1 d_rdlvl_load

Signal: dfi_rdlvl_load

Full Name: DFI Read Level Load

Direction: MC → PHY

Width: 1 bit (global or per rank)

Function: Commands the PHY to load the current read levelling delay results from the training registers into the active delay line registers. This is the *commit* step: training finds the optimal delays, but they are held in shadow registers until dfi_rdlvl_load is asserted, at which point all delay codes go live atomically.

Why Separate Load? Training may iterate several times (coarse scan, fine scan, margin verification). Only when the MC is satisfied does it issue the load to lock in the values without disrupting ongoing traffic.

7.1.2 d_rdlvl_cs_n

Signal: dfi_rdlvl_cs_n

Full Name: DFI Read Level Chip Select

Direction: MC → PHY

Width: 1 bit per rank

Function: Selects which rank is being trained during the read levelling procedure. Only the selected rank responds to read levelling commands. This allows the PHY to train each rank independently, since each rank has different flight times from the PHY.

7.1.3 d_rdlvl_en

Signal: `dfi_rdlvl_en`

Full Name: DFI Read Level Enable

Direction: MC → PHY

Width: 1 bit

Function: Enables the PHY's read levelling (per-bit RX centering) training engine. While asserted:

- The PHY activates its RX delay sweep logic
- Incoming read data is compared against the internal pattern generator
- Pass/fail results are accumulated per bit, per lane
- The PHY drives `dfi_rdlvl_delay_X` (PHY→MC) with the current delay codes

Normal operation resumes when `dfi_rdlvl_en` is deasserted.

7.1.4 `d_rdlvl_edge`

Signal: `dfi_rdlvl_edge`

Full Name: DFI Read Level Edge

Direction: MC → PHY

Width: 1 bit

Function: Selects which DQS edge (rising or falling) is being swept during read levelling:

- `dfi_rdlvl_edge` = 0: Train the **rising-edge** sample point (odd DQ bits in each UI)
- `dfi_rdlvl_edge` = 1: Train the **falling-edge** sample point (even DQ bits)

Since DDR is double-data-rate, each DQ bit is sampled on both edges. Each edge has an independent delay path in the PHY and must be trained separately. This signal selects which half of the eye is being measured.

7.1.5 `d_rdlvl_delay_X`

Signal: `dfi_rdlvl_delay_X`

Full Name: DFI Read Level Delay (per bit/lane)

Direction: MC → PHY (writes delay codes to shadow registers); also PHY → MC (reports current codes)

Width: Typically $5 \times 8 = 40$ bits (5-bit code per DQ bit $\times$ 8 bits per lane); varies by implementation

Function: This is the delay code register interface.

The MC can:

1. **Read** (PHY→MC): the current delay code for each DQ bit (to observe training progress)
2. **Write** (MC→PHY): force a specific delay code into the shadow registers (for firmware-directed training or debug)

The `_X` in the name indicates multiple instances — one per DQ bit within the lane, so for a $\times 8$ lane there are 8 independent delay codes.

Code Resolution: Each delay code step = $\Delta_s \approx 5$ ps. A 7-bit code covers 0–127 steps $\times 5$ ps = 0–635 ps of delay compensation range.

7.1.6 d_rdlvl_gate_en

Signal: dfi_rdlvl_gate_en
Full Name: DFI Read Level Gate Enable
Direction: MC $\rightarrow$ PHY
Width: 1 bit
Function: Enables specifically the DQS gate training sub-step of read levelling. When asserted:

- The PHY's gate position sweep logic is activated
- The PHY sweeps the gate open-position across DQS preamble detection window
- Pass/fail is recorded for each gate position tested
- The PHY drives dfi_rdlvl_gate_delay_X with optimal gate codes

Distinction from dfi_rdlvl_en: dfi_rdlvl_gate_en trains *when* to open the DQS gate (coarse, $\sim t_{CK}$ granularity). dfi_rdlvl_en trains *where* inside the eye to sample (fine, ps granularity). Gate training **must complete before** per-bit read levelling can begin.

7.1.7 d_rdlvl_gate_delay_X

Signal: dfi_rdlvl_gate_delay_X
Full Name: DFI Read Level Gate Delay
Direction: Bidirectional (MC writes; PHY reports)
Width: Typically 5–7 bits per lane (one gate per DQS pair, not per bit)
Function: The delay code for the DQS gate position of each byte lane. Physically, this adjusts when (in CK cycles and sub-cycle) the gating flip-flop opens relative to the expected DQS preamble.

$$t_{gate_open} = n_{CK} \times t_{CK} + k_{gate} \times \Delta_s$$

where n_{CK} is an integer number of clocks (coarse) and k_{gate} is the fine delay code.

Gate Training Result: After gate training completes, this register holds the code that places the gate open event exactly at the DQS preamble onset, with maximum margin to both the Hi-Z boundary and the first DQS edge.

7.2 Write Levelling Signals

7.2.1 d_wrlvl_load

Signal: dfi_wrlvl_load
Full Name: DFI Write Level Load
Direction: MC $\rightarrow$ PHY
Width: 1 bit
Function: Identical in concept to dfi_rdlvl_load but for the write levelling delay results.

Asserted by the MC after write levelling training is complete to atomically load the DQS TX delay codes from shadow registers to the active delay lines.

Write Levelling Reminder: Write levelling finds the DQS-to-CK phase offset at each DRAM (caused by fly-by topology). The optimal DQS launch delay per lane is stored in a shadow register during training; `dfi_wrlvl_load` commits it to hardware.

7.2.2 `d_wrlvl_cs_n`

Signal: `dfi_wrlvl_cs_n`
Full Name: DFI Write Level Chip Select
Direction: MC → PHY
Width: 1 bit per rank
Function: Selects the rank to train during write levelling. Since each DRAM rank has a different physical location on the fly-by CK bus, each rank sees a different CK phase relative to DQS and needs an independent write levelling delay code.

7.2.3 `d_wrlvl_strobe`

Signal: `dfi_wrlvl_strobe`
Full Name: DFI Write Level Strobe
Direction: MC → PHY
Width: 1 bit
Function: Commands the PHY to pulse DQS once (a single rising edge) towards the DRAM during write levelling mode. Each strobe pulse:

1. Causes the DRAM (in write levelling mode) to sample CK at the DQS rising edge
2. The DRAM drives `DQ[0]` = sampled CK value back to the PHY
3. The PHY captures `DQ[0]` and reports it via a status signal to the MC
4. The MC increments the delay code and re-asserts `dfi_wrlvl_strobe`

Rate: The strobe should be pulsed no more often than t_{WLS} (write level strobe period, typically ≥ 10 ns) to allow DRAM to respond and PHY to capture.

7.2.4 `d_wrlvl_en`

Signal: `dfi_wrlvl_en`
Full Name: DFI Write Level Enable
Direction: MC → PHY
Width: 1 bit
Function: Enables the PHY's write levelling engine. When asserted:

- PHY places the DRAM in write levelling mode via `MR1[A7]=1`
- PHY activates the DQS TX delay sweep mechanism
- `DQ[0]` feedback from DRAM is captured by the PHY's RX circuitry
- PHY reports feedback to MC via PHY status signals

Deasserted when write levelling is complete. The PHY exits write levelling mode (MR1[A7]=0) automatically.

7.2.5 d_wrlvl_delay_X

Signal: dfi_wrlvl_delay_X

Full Name: DFI Write Level Delay

Direction: Bidirectional (MC writes; PHY reports)

Width: Typically 7 bits per lane (one DQS TX delay code per byte lane)

Function: The DQS TX delay code for each byte lane determined by write levelling. This adjusts when in time (sub-cycle precision) the PHY launches the DQS rising edge.

$$t_{DQS_launch} = t_{nominal} + k_{WL,i} \times \Delta_s$$

where $k_{WL,i}$ is the write level delay code for lane i .

After Training: $k_{WL,i}$ is set to the code at which DQ[0] feedback transitions 0→1, indicating DQS rising edge is co-incident with the DRAM's CK rising edge at lane i .

8 Low Power Control Interface

Interface Group: Low Power Control Interface

The Low Power Interface enables the MC and PHY to coordinate power-saving states for the PHY itself (distinct from DRAM power states controlled via CKE). When the memory bus is idle for extended periods, the PHY can shut down internal blocks (PLL, DLL, IO drivers) to save power. A handshake protocol ensures the PHY can always wake up in time to service the next access.

Direction: Bidirectional handshake

8.1 d_lp_req

Signal: dfi_lp_req

Full Name: DFI Low Power Request

Direction: MC → PHY

Width: 1 bit per interface (control / data)

Function: The MC asserts dfi_lp_req to request the PHY to enter a low power state. The DFI specification defines two independent LP request lines:

- d_lp_ctrl_req: Low power request for the control interface (command/address PHY blocks)
- d_lp_data_req: Low power request for the data interface (DQ/DQS PHY blocks)

What the PHY Powers Down: DLL (if not needed for data integrity), IO driver bias circuits, RX input amplifiers, internal clock distribution (if CK is disabled).

Important: The PHY must respond with dfi_lp_ack = 1 (PHY→MC) before actually entering the low power state. The MC must not issue any memory transactions after asserting LP_req until it sees LP_ack deasserted (PHY is awake again).

8.2 d_lp_wakeup

Signal: dfi_lp_wakeup

Full Name: DFI Low Power Wakeup

Direction: MC → PHY

Width: 4 bits (encodes the anticipated wakeup time budget)

Function: Tells the PHY in advance how long the low power period will last. The PHY uses this to decide how deeply to power down.

d_lp_wakeup	Wakeup Budget	PHY LP Depth	Blocks Saved
4'b0001	16 clocks	Shallow: gate clocks	IO bias
4'b0010	32 clocks	Medium	IO + RX amps
4'b0011	64 clocks	Deep: DLL off	IO + RX + DLL
4'b0100	128 clocks	Very Deep	+ clock tree
4'b0101	256 clocks	Max depth + PLL	(full power-off)
...	...	...	...

Key Trade-off: Deeper power-down saves more power but requires a longer wakeup time (PLL re-lock, DLL re-lock, IO re-bias). The MC encodes the expected idle duration so the PHY can make the optimal power-vs-wakeup trade-off automatically.

Wakeup Sequence:

1. MC deasserts `dfi_lp_req` = 0 (wakeup trigger)
2. PHY begins re-initialising its powered-down blocks
3. PHY deasserts `dfi_lp_ack` = 0 only after it is fully operational
4. MC waits for `dfi_lp_ack` = 0 before issuing any command

9 Complete Signal Quick-Reference Table

Table 2: All DFI PHY Input Interface Signals — Complete Reference

Signal	Dir.	Width	Function Summary
CONTROL INTERFACE			
dfi_address_pN	MC→PHY	14–18 b	Row/col/MR address; driven onto DRAM A[13:0] pins
dfi_bank_pN	MC→PHY	2 b	Bank select (BA[1:0]); targets one of 4 banks per BG
dfi_cas_n_pN	MC→PHY	1 b	CAS# command bit; with RAS#/WE# forms command word
dfi_cke_pN	MC→PHY	1 b/rank	Clock enable; controls DRAM power-down/self-refresh states
dfi_cs_n_pN	MC→PHY	1 b/rank	Chip select; DRAM only accepts commands when CS#=L
dfi_odt_pN	MC→PHY	1 b/rank	ODT enable; dynamically switches on-die termination resistors

(continued on next page)

Signal	Dir.	Width	Function Summary
dfi_ras_n_pN	MC→PHY	1 b	RAS# command bit (DDR3); repurposed as A16 in DDR4
dfi_reset_n_pN	MC→PHY	1 b	Async DRAM reset; immediately tristates DRAM; $\geq 200 \mu\text{s}$ at POR
dfi_we_n_pN	MC→PHY	1 b	WE# command bit; distinguishes READ/WRITE, selects precharge mode
WRITE DATA INTERFACE			
dfi_wrdata_pN	MC→PHY	16 b/slice	Write payload; bits[7:0]=rising DQS data, [15:8]=falling DQS data
dfi_wrdata_en_pN	MC→PHY	1 b/slice	Write burst enable; PHY drives DQ/generates DQS only when HIGH
dfi_wrdata_mask_pN	MC→PHY	2 b/slice	Write data mask; suppresses DRAM store for masked byte on that edge
READ DATA INTERFACE			
dfi_rddata_en_pN	MC→PHY	1 b/slice	Read enable look-ahead; tells PHY to open DQS gate and capture DQ
dfi_rddata_pN	PHY→MC	16 b/slice	(PHY output) Captured read data returned to MC
dfi_rddata_valid_pN	PHY→MC	1 b/slice	(PHY output) Asserted when rddata is valid; MC must check before consuming
UPDATE INTERFACE			
dfi_ctrlupd_req	MC→PHY	1 b	MC requests PHY apply pending config update; awaits ctrlupd_ack
dfi_ctrlupd_ack	PHY→MC	1 b	(PHY output) PHY acknowledges controller update completion
dfi_phyupd_req	PHY→MC	1 b	(PHY output) PHY requests pause for self-initiated update (DLL re-lock, ZQ)
dfi_phyupd_ack	MC→PHY	1 b	MC grants PHY permission to proceed with its update; traffic paused
STATUS INTERFACE			
dfi_data_byte_disable	MC→PHY	1 b/slice	Permanently disables a byte lane (tristate DQ/DQS; exclude from training)
dfi_dram_clk_disable	MC→PHY	1 b/rank	Stops CK/CK# output; used during self-refresh, power-gating, PLL change
dfi_freq_ratio	MC→PHY	2 b	DFI:DRAM clock ratio (01=1:1, 10=1:2, 11=1:4)
dfi_init_start	MC→PHY	1 b	Triggers PHY DRAM init and full training sequence
dfi_init_complete	PHY→MC	1 b	(PHY output) Training complete; MC may begin scheduling
dfi_parity_in_pN	MC→PHY	1 b/rank	Odd parity over CA bus; forwarded to DRAM PAR pin for CA error detection
TRAINING INTERFACE — READ LEVELLING			
dfi_rdlvl_load	MC→PHY	1 b	Commits read levelling delay codes from shadow to active registers
dfi_rdlvl_cs_n	MC→PHY	1 b/rank	Selects rank to train during read levelling
dfi_rdlvl_en	MC→PHY	1 b	Enables per-bit RX delay centering engine in PHY

(continued on next page)

Signal	Dir.	Width	Function Summary
dfi_rdlvl_edge	MC→PHY	1 b	Selects rising (0) or falling (1) DQS edge for levelling
dfi_rdlvl_delay_X	Bidir	5–7 b/bit	Per-DQ-bit RX delay code; MC writes to shadow, PHY reports current
dfi_rdlvl_gate_en	MC→PHY	1 b	Enables DQS gate position training in PHY
dfi_rdlvl_gate_delay_X	Bidir	5–7 b/lane	Per-lane DQS gate open position code
TRAINING INTERFACE — WRITE LEVELLING			
dfi_wrlvl_load	MC→PHY	1 b	Commits write levelling DQS TX delay codes from shadow to active
dfi_wrlvl_cs_n	MC→PHY	1 b/rank	Selects rank to train during write levelling
dfi_wrlvl_strobe	MC→PHY	1 b	Triggers one DQS pulse; DRAM samples CK and returns DQ[0] feedback
dfi_wrlvl_en	MC→PHY	1 b	Enables write levelling engine; PHY enters DRAM WL mode (MR1[A7]=1)
dfi_wrlvl_delay_X	Bidir	7 b/lane	Per-lane DQS TX delay code found by write levelling
LOW POWER CONTROL INTERFACE			
dfi_lp_req	MC→PHY	1 b	Requests PHY enter low power state; awaits lp_ack
dfi_lp_wakeup	MC→PHY	4 b	Encodes expected idle duration; PHY selects LP depth accordingly
dfi_lp_ack	PHY→MC	1 b	(PHY output) PHY confirms LP state entered / confirms exit complete

10 Signal Interaction: Key Timing Diagrams

10.1 Write Transaction Timing

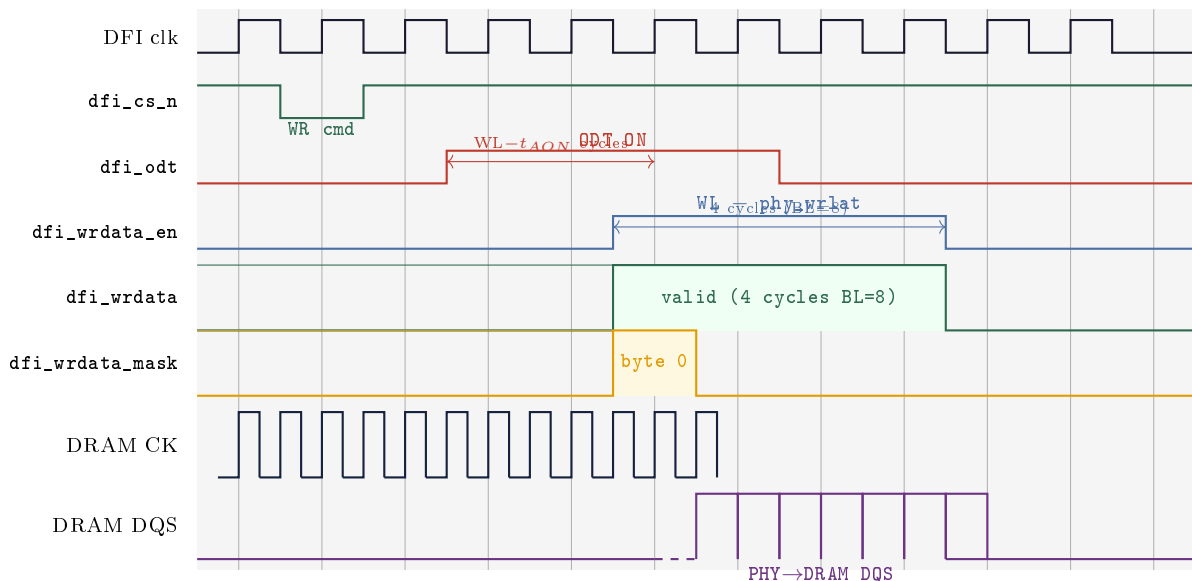


Figure 2: Write transaction DFI timing. ODT is asserted before the data window; `wrdata_en` gates the burst; PHY generates DQS preamble and postamble automatically.

10.2 Init + Training Handshake

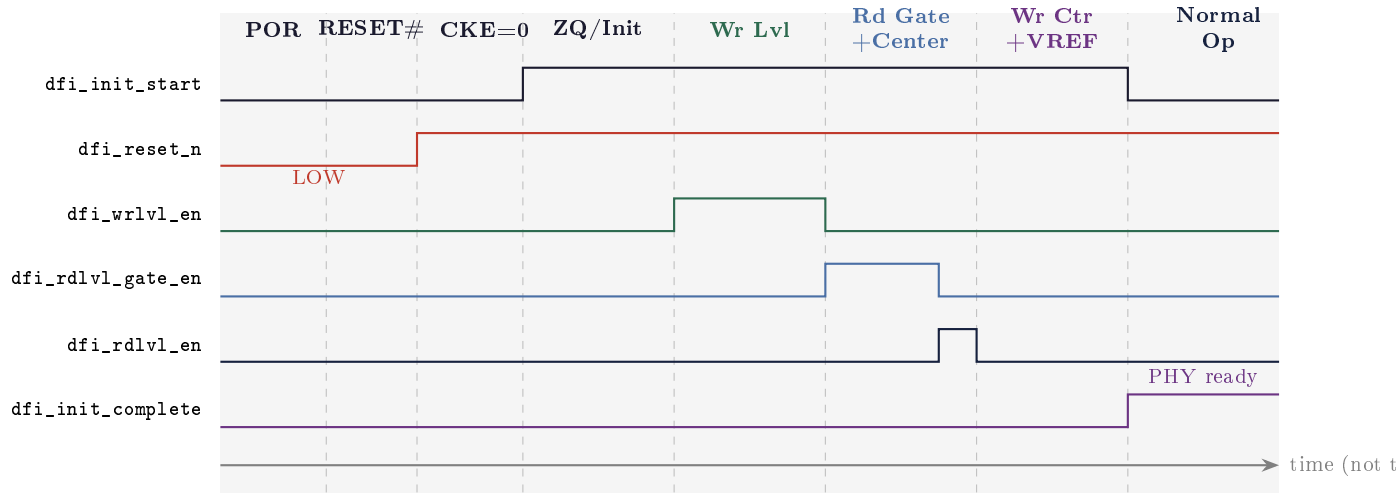


Figure 3: Initialisation and training sequence. `dfi_init_start` is held high during the entire training process. Each training engine is enabled sequentially. `dfi_init_complete` (PHY output) signals readiness for normal operation.

11 Summary and Key Takeaways

Key Takeaways — DFI PHY Input Interface

1. **DFI decouples the MC from the PHY:** any DFI-compliant MC pairs with any DFI-compliant PHY.
2. **Control Interface (9 signals):** carries address, bank, and command. All are MC→PHY. PHY applies a pipeline latency before driving DRAM pins.
3. **Write Data Interface (3 signals):** `dfi_wrdata` carries payload; `dfi_wrdata_en` gates the burst; `dfi_wrdata_mask` enables byte masking. The PHY serialises parallel DFI data to high-speed DQ.
4. **Read Data Interface:** `dfi_rddata_en` is a look-ahead signal sent before data arrives. The PHY returns data via `dfi_rddata` with a `dfi_rddata_valid` qualifier.
5. **Update Interface:** Two independent handshakes — controller-initiated (`ctrlupd`) and PHY-initiated (`phyupd`). Ensure atomic config updates without data corruption.
6. **Status Interface:** `dfi_freq_ratio` is critical — it tells the PHY how many DFI clocks per DRAM CK, affecting every timing calculation. `dfi_init_start` triggers the full training boot sequence.
7. **Training Interface** is the richest group: separate enable, chip-select, strobe, delay-code, load, and edge signals for both read levelling and write levelling. Training is split into coarse (gate) and fine (per-bit) steps.
8. **Low Power Interface:** `dfi_lp_wakeup` encodes idle duration so the PHY selects optimal power-down depth vs. wakeup latency trade-off.
9. **Per-slice (`_pN`) signals** exist because each byte lane is physically independent with its own δ_i skew and independent delay compensation.

Common Pitfalls

- `dfi_rddata_en` is MC→PHY; `dfi_rddata` is PHY→MC — don't confuse directions.
- `dfi_phyupd_ack` is MC→PHY even though it says “PHY” — it's the MC granting the PHY's request.
- `dfi_wrlvl_strobe` triggers one DQS pulse only — it is not a continuous signal.
- `dfi_rdlvl_gate_en` and `dfi_rdlvl_en` are sequential steps, not simultaneous.
- `dfi_lp_wakeup` is sent *before* asserting `dfi_lp_req`, not after.
- `dfi_data_byte_disable` is static (set once at boot); `dfi_dram_clk_disable` is dynamic.